CETEC UBATIC 2026-27

Grupo 2: Qdrant + CLIP/CONCH + LangGraph/ADK

Informe Técnico - Etapa 2

Baseline, LLM como Juez, DSPy GEPA y Mejora de la Recuperación

Mateo Gonzalez Pautaso - Alen Calandria

Noviembre 2026

1. Contexto y Objetivo de la Etapa 2

Al cierre de la Etapa 1, el asistente quedó integrado como un agente LangGraph sobre Qdrant, con búsqueda híbrida (vectorial, por entidades y por palabras clave), embeddings de texto MiniLM y embeddings de imagen UNI y PLIP. El sistema medía la recuperación, pero no tenía una forma reproducible de medir la calidad de las respuestas que genera.

Los objetivos de la Etapa 2 fueron:

- Tarea 1: Construir un baseline reproducible que corra el pipeline completo sobre el golden set de 32 preguntas y mida el recall fuente+página@5.
- Tarea 2: Implementar un LLM como juez, con una rúbrica docente de cuatro dimensiones, para puntuar las respuestas y no solo los fragmentos recuperados.
- Tarea 3: Optimizar la instrucción de respuesta con DSPy GEPA, usando el juez como métrica.
- Tarea 4: Mejorar la recuperación (chunking y búsqueda), que GEPA identificó como el límite del sistema.

Además, hubo que migrar los modelos del pipeline, porque Groq retiró del plan gratuito los dos modelos que usaba el sistema.

2. Cambios de Infraestructura

2.1 Migración de Modelos

Groq retiró Llama-4-Scout el 17/07/2026 y Llama 3.3 70B el 16/08/2026. Desde esas fechas, las llamadas devolvían el error 404 model_not_found. Los reemplazos se eligieron entre los modelos que el plan gratuito sigue ofreciendo:

Rol	Antes	Ahora
Generador (texto e imagen)	Llama-4-Scout	qwen/qwen3.8-27b (multimodal)
Juez	Llama 3.3 70B	openai/gpt-oss-120b
Reflexión de GEPA	(no existía)	openai/gpt-oss-120b

Qwen razona antes de responder, lo que gasta cuota sin aportar a las respuestas del RAG. Por eso se envía con `reasoning_effort=none` y `reasoning_format=hidden`, y se elimina cualquier bloque `<think>` que quede en la respuesta. El modelo se configura con la variable `LLM_MODELO`.

El script `optimizacion/modelos.py` lista los modelos que ve cada key y, con `--probar`, hace una llamada de texto, una con imagen, una al juez y una al modelo de reflexión. Sirve para detectar un modelo retirado antes de correr una evaluación larga.

2.2 Rotación de API Keys y Límites del Plan Gratuito

La clase `RotadorClaves` (`src/claves.py`) reparte las llamadas entre las keys de `GROQ_API_KEYS` por turnos. Cuando una key recibe un límite de uso (429), queda en pausa el tiempo que indica Groq ("Please try again in 7.5s") y la llamada sigue con la siguiente. Si todas están en pausa y la primera se libera en menos de 90 segundos (`GROQ_ESPERA_MAX_S`), espera en lugar de fallar, y sigue reintentando mientras la llamada no lleve más de 10 minutos (`GROQ_ESPERA_TOTAL_S`). Al principio cortaba a las cuatro rondas, y en la segunda corrida de GEPA eso convirtió un límite por minuto en una cuota agotada (sección 6.7).

Groq cuenta los límites por organización, no por key: dos keys de la misma cuenta comparten la cuota. Para que la rotación sirva, las keys tienen que ser de cuentas distintas. Los límites que más afectaron el trabajo fueron:

Límite	Valor	Efecto en el proyecto
Tokens por día, por modelo y cuenta	200.000	Una corrida de GEPA con --budget light agotó al juez en 3 horas con una sola cuenta
Tokens de entrada por minuto de Qwen	7.000	El pedido de extracción del temario (7.760 tokens) se rechazaba con 413; la muestra se redujo a 12.000 caracteres
Tokens de salida por minuto de Qwen	1.000	Groq rechaza pedidos con max_tokens mayor; GEPA usa 900

Un 413 ("Request too large") ya no pausa la key: el pedido excede el límite de cualquier key, así que rotar solo dejaría todas en pausa.

2.3 Renderizado de PDF sin Poppler

El renderizado de páginas pasó a usar PyMuPDF (`get_pixmap`), con pdf2image como respaldo. En Windows ya no hace falta instalar Poppler ni Tesseract para indexar los manuales.

3. Tarea 1: Baseline sobre el Golden Set

3.1 Golden Set y Métrica

El golden set creció de 30 preguntas en el Sprint 2 a 32: 20 sobre arch2, 5 sobre arch3 y 7 sobre arch4. Cada pregunta tiene una respuesta de referencia extraída del manual, la fuente esperada y las páginas donde está la respuesta.

La métrica de recuperación es el recall fuente+página@5: la fracción de pares (fuente, página) esperados que aparecen entre los 5 primeros fragmentos recuperados. Es más estricta que el Recall@K del Sprint 2, porque la página tiene que venir del PDF correcto.

El script `optimizacion/baseline.py` corre el pipeline completo pregunta por pregunta y guarda la traza de cada una (contexto recuperado, respuesta y referencias) en un

JSONL. Si una pregunta falla, la siguiente corrida la vuelve a ejecutar sin repetir las demás.

3.2 Regresión de Recall por Entidades Inferidas

Las primeras corridas con Qwen dieron un recall de 0.22. El diagnóstico encontró dos causas:

- Qwen agregaba entidades que no estaban en la consulta. En la búsqueda por palabras clave, esas entidades puntuaban 0.95 y desplazaban a los fragmentos relevantes.
- Los fragmentos de arch2 no tenían dominio asignado, porque las reglas de entidades no cubrían cartílago, tejido óseo, tejido muscular ni tejido nervioso.

La corrección tiene dos partes. Una entidad que propone el LLM solo se conserva si aparece en la consulta y no la detectaron ya las reglas. Y se agregaron los cuatro dominios de arch2 a `REGLAS_ENTIDADES`. En una simulación de la búsqueda por palabras clave, el recall subió de 0.844 a 0.893; en el baseline completo (v1) quedó en 0.872.

3.3 Resultados

El baseline v2, sobre el que se corrió GEPA, obtuvo un recall fuente+página@5 de 0.852:

Fuente	Preguntas	Recall fuente+página@5
arch2.pdf	20	0.825
arch3.pdf	5	0.850
arch4.pdf	7	0.929
Total	**32**	**0.852**

La fuente dominante entre los fragmentos recuperados fue la correcta en el 96.9 % de las preguntas. Cada pregunta tardó 14.9 segundos en promedio, con el pipeline completo.

4. Tarea 2: LLM como Juez

4.1 Rúbrica

El juez (`optimizacion/juez.py`, con gpt-oss-120b) recibe la pregunta, la respuesta de referencia, el contexto recuperado y la respuesta del sistema. Devuelve un JSON con un puntaje entero de 1 a 5 y una justificación para cada dimensión:

Dimensión	Qué mide	Peso
Corrección	Coincide con la respuesta de referencia del manual	0.35
Fidelidad	Todo lo afirmado está sustentado en el contexto recuperado	0.30
Notación	Terminología y notación histológica correctas y precisas	0.20
Estilo docente	Explica como lo haría un docente de la cátedra	0.15

El puntaje global va de 0 a 1. Cada dimensión se normaliza y se promedia con su peso:

```
global = Σ peso_d · (puntaje_d − 1) / 4  /  Σ pesos
```

Además del juez, el script verifica las citas sin LLM: si la respuesta cita una fuente, si la fuente citada está en el contexto y si es la fuente esperada.

4.2 Ajustes entre el Baseline v1 y el v2

Las justificaciones del juez sobre el baseline v1 mostraron dos problemas, que se corrigieron antes del v2. Ninguno de los dos cambios toca la recuperación:

- Prompt en modo texto: el prompt incluía un bloque de metadatos de búsqueda (TÉRMINOS, ENTIDADES, TEMA y ANÁLISIS VISUAL). Con ese bloque, Qwen respondía "el manual no contiene..." aunque el contexto sí tenía la respuesta. Se sacó del modo texto; el modo multimodal quedó igual (se verificó que el prompt multimodal sea idéntico en 32 combinaciones).
- Golden set: la referencia de la pregunta #22 (conductos de Havers y Volkmann) se reescribió, porque el manual no describe la diferencia entre ambos conductos: solo los nombra como estructuras del hueso compacto (páginas 9 y 16). En la #31 (células ependimarias) se corrigieron las páginas de referencia a 30, 31 y 35.

4.3 Resultados

El puntaje global subió de 0.729 a 0.788. La mayor mejora fue en fidelidad (+0.41) y corrección (+0.25):

Métrica	Baseline v1	Baseline v2	Δ
Recall fuente+página@5	0.872	0.852	−0.020
Puntaje global del juez (0–1)	0.729	0.788	+0.060
Corrección (1–5)	3.47	3.72	+0.25
Fidelidad (1–5)	4.25	4.66	+0.41
Notación (1–5)	4.59	4.72	+0.13
Estilo docente (1–5)	3.38	3.41	+0.03

Pregunta por pregunta, el v2 es mejor en 14, peor en 7 e igual en 11. Por fuente, el puntaje global del v2 fue 0.771 en arch2, 0.805 en arch3 y 0.827 en arch4. El 100 % de las respuestas cita una fuente, la fuente citada está en el contexto y es la esperada.

La diferencia de recall entre v1 y v2 se explica sobre todo por las páginas de referencia corregidas en #22 y #31, ya que entre ambas corridas no se modificó la recuperación.

5. Tarea 3: Optimización del Prompt con DSPy GEPA

5.1 Configuración

GEPA (`optimizacion/gepa.py`) evoluciona la instrucción de respuesta. En cada iteración evalúa un candidato sobre 3 preguntas de entrenamiento, le pasa las justificaciones del juez a un modelo de reflexión, y este propone una instrucción nueva. Si la nueva mejora esas 3 preguntas, se evalúa sobre todo el conjunto de validación.

Parámetro	Valor
Modelo de respuesta	qwen/qwen3.8-27b, el mismo de producción
Juez y reflexión	openai/gpt-oss-120b
Train / validación	22 / 10 preguntas del golden set (semilla 0)
Contexto	El recuperado en el baseline v2, truncado a 4.000 caracteres como en producción
Presupuesto	100 evaluaciones de la métrica (se usaron 90)
Métrica	Puntaje global del juez, con sus justificaciones como feedback
Duración	10 minutos, con 3 keys de Groq

5.2 Protección de la Cuota

GEPA y `dspy.Evaluate` convierten cualquier excepción en un puntaje 0 y siguen. Si la cuota se agota a mitad de la corrida, GEPA seguiría optimizando contra ceros y el prompt resultante no valdría nada. Esto pasó en la primera corrida con `--budget light`: duró 3 horas y agotó la cuota diaria del juez.

Para evitarlo se agregaron cuatro protecciones:

- El primer límite de uso que la rotación no resuelve marca la corrida. Las llamadas siguientes fallan sin ir a la API y el resultado no se guarda.
- DSPy ya no reintenta por su cuenta (`num_retries=0`): sus reintentos dormían el tiempo de espera de Groq, que con la cuota diaria agotada son minutos por llamada. Los 429 los resuelve el rotador.
- Las respuestas que vuelven del caché de DSPy se normalizan (de AttributeDict a dict), porque GEPA las rechazaba al retomar una corrida. Así, una corrida cortada retoma casi donde quedó.
- La respuesta del generador se limita a 900 tokens (`GEPA_MAX_TOKENS_TAREA`), por debajo del límite de salida de Qwen, y un pedido rechazado por tamaño corta la corrida con su propio mensaje.

5.3 Resultados

GEPA no encontró un prompt mejor. El mejor programa que devolvió es la semilla misma, con 0.922 en validación, y ninguno de los 6 candidatos la superó:

Programa	Puntaje en validación (0–1)
Semilla (prompt actual)	**0.922**
Candidato 2	0.918
Candidato 1	0.906
Candidato 6	0.873
Candidato 4	0.854
Candidato 3	0.848
Candidato 5	0.840

Hay tres razones para que no mejore:

- Casi no queda margen: la semilla saca 0.922, y 8 de las 10 preguntas de validación están entre 0.9 y 1.0.
- La pregunta que falla depende de la recuperación. En la #30 (oligodendrocitos), la semilla saca 0.54 y los candidatos entre 0.0 y 0.2. El fragmento recuperado se corta antes de la función de la célula, y ningún prompt puede compensarlo.

- Los prompts más largos empeoran esa pregunta. GEPA agregó reglas sobre cuándo decir que la información no está, y con ellas Qwen responde que el manual no la trae.

Además, con 10 preguntas de validación cada una pesa un 10 % del promedio, así que diferencias de 0.01 entre prompts son ruido. La decisión es mantener el prompt actual: `PROMPTS_OPTIMIZADOS` queda en `false`, y el trabajo siguiente se concentró en la recuperación (sección 6). Con la recuperación nueva, la #30 saca 1.00, y GEPA se volvió a correr sobre el baseline v6 (sección 6.7).

6. Tarea 4: Mejora de la Recuperación

Las 6 preguntas con puntaje global menor a 0.6 en el baseline v2 fallaban por recuperación, no por el prompt: el fragmento con la respuesta no llegaba al modelo, o llegaba cortado. La recuperación se mejoró en cuatro pasos, cada uno medido con una corrida completa del baseline y del juez (v3 a v6).

6.1 Chunking por Oraciones, Fichas y Secciones (v3 y v4)

Antes, cada página se cortaba cada 500 caracteres sin mirar el contenido. El corte caía a mitad de oración o de lista ("Por el número de prolongaciones p" | "ueden ser: unipolares..."), y la respuesta quedaba repartida entre dos fragmentos. El módulo nuevo `src/chunking.py` agrupa las líneas de cada página en segmentos que no conviene partir (una oración, un ítem de lista, una línea de ficha como "Laminilla No: Laringe 43") y arma los fragmentos con segmentos enteros:

- Cada fragmento apunta a 650 caracteres y nunca pasa de 850, porque MiniLM lee hasta 256 tokens (unos 750 caracteres de español). Uno de menos de 300 caracteres no se cierra: se le suma el segmento siguiente.
- "Imagen N" y "Práctica N" empiezan un fragmento nuevo, y los fragmentos que siguen a una ficha o práctica repiten su título.
- Los encabezados de la plantilla de práctica (Objetivos, Fundamento teórico, Materiales, Procedimiento, Resultados) también empiezan un fragmento (v4). Así una definición no comparte fragmento, ni embedding, con la lista de materiales.
- Una frase que presenta una lista ("cada lobulillo está compuesto por:") o un encabezado corto ("Oligodendrocitos.") nunca cierra un fragmento: va con lo que sigue.

El contexto que recibe el modelo pasó de 4.000 a 6.000 caracteres (`MAX_CONTEXTO_PROMPT`), para que entren 6 fragmentos completos. La marca de indexación guarda la versión del chunking (`CHUNKING_VERSION`): si cambia, el próximo arranque vacía la colección de fragmentos y reindexa, sin costo de LLM.

El v3 resolvió las tres preguntas cuyo fragmento se cortaba: #27 (0.00 → 0.89), #30 (0.54 → 1.00) y #31 (0.10 → 0.96). El recall no cambió (0.867), porque la página correcta ya se recuperaba; lo que cambió es que el fragmento recuperado contiene la respuesta entera.

6.2 Diagnóstico por Pregunta

El v4 bajó el puntaje global del juez respecto del v3, casi todo por la pregunta #1 (lámina elástica interna), que pasó de 1.00 a 0.10 con el mismo recall. Para saber si un error es de recuperación o de generación se agregó `optimizacion/inspeccionar.py`. Sin llamar a ningún modelo, muestra por pregunta los fragmentos que llegaron al modelo (fuente, página, puntaje y comienzo del texto), qué fracción de las palabras de la referencia está en el contexto, la respuesta y la justificación del juez.

En la #1, solo 3 de las 16 palabras de la referencia estaban en el contexto: el único fragmento que contiene "lámina elástica interna" no había llegado. Qwen respondió bien que el manual no lo describía, a partir de lo que recibió. El problema estaba en la búsqueda.

6.3 Búsqueda Coseno + BM25 (v5)

La búsqueda de texto sumaba los puntajes de tres listas de 10 resultados: la vectorial, la de entidades (que devuelve los primeros 10 fragmentos que coinciden, sin ningún orden) y la de palabras clave, que solo se activaba con términos de varias palabras o con resultados vectoriales flojos. Un fragmento que aparecía en una sola lista perdía siempre contra los que aparecían en dos. En la #1, el fragmento exacto sumaba 0.95 por palabras clave y los de arriba alrededor de 1.5. En la #15 (laringe 43) y la #21 (osteona), la búsqueda por palabras clave ni se activaba.

En las consultas sin imagen, la búsqueda nueva puntúa todos los fragmentos del manual (unos cien) con dos señales:

```
puntaje = peso · coseno(MiniLM) + (1 − peso) · BM25 / BM25 del mejor fragmento
```

BM25 (`src/lexico.py`) pesa cada término por su rareza en el manual: "osteona", "laringe" o "43" deciden el orden, y "tejido" o "arteria" casi no suman. Busca con la pregunta original y con la reescrita, porque la reescritura a veces pierde un término puntual. Como se calcula sobre todos los fragmentos, ninguno pierde por aparecer en una sola búsqueda. La búsqueda multimodal (con imagen) no cambió.

El peso se eligió con `optimizacion/recuperacion.py`, que mide el recall de la búsqueda sola con distintos pesos y los embeddings reales, sin llamar a ningún LLM:

Peso del coseno	Total	arch2	arch3	arch4
1.0 (solo embeddings)	0.727	0.825	0.417	0.667
0.0 a 0.7	0.927	0.983	0.767	0.881

Entre 0.0 y 0.7 el recall es el mismo: MiniLM está entrenado en inglés y en español el coseno varía poco de un fragmento a otro, así que BM25 decide el orden y el coseno desempata. Se dejó 0.5 (`HIBRIDA_PESO_VECTOR`), en el medio de ese rango: el golden set usa el vocabulario del manual, y un estudiante que pregunte con otras palabras solo cuenta con el coseno.

El v5 subió el puntaje global del juez +0.10 sobre el v4. Las preguntas que más subieron son las que fallaban por búsqueda: #0 (0.14 → 1.00), #1 (0.10 → 1.00), #5 (0.19 → 0.75) y #15 (0.10 → 0.89).

6.4 Encabezados entre Páginas (v6)

Cada página se divide por separado, porque cada fragmento guarda su página. Pero el texto sigue de una página a otra: la lista "Hueso compacto: Células." empieza en la página 8 de arch2 y los osteocitos y osteoclastos quedan en la 9, sin esas palabras. En la #20 (tipos de células del hueso compacto), BM25 prefería tres fichas de imagen que repiten "hueso compacto".

Ahora, cuando una página continúa la anterior, sus fragmentos arrancan con el título de la práctica y, si sigue una lista, con su encabezado. En el v6 la página 9 entra cuarta y la #20 sube de 0.47 a 1.00. El resto de las preguntas queda igual dentro del ruido.

6.5 Corrección del Golden Set

Con la recuperación resuelta, `inspeccionar.py` mostró que 5 respuestas de referencia no salían del manual: venían de un libro de texto o lo contradecían. Una respuesta fiel al manual sacaba un puntaje bajo:

#	Pregunta	Problema de la referencia anterior
5	Organización del tejido testicular	Incluía vasos en el intersticio, que el manual no nombra; el intersticio se describe en la página 2, que no estaba entre las esperadas
9	Espermatogonia A oscura	Decía "cromatina más condensada u oscura"; el manual describe una rarefacción de la cromatina en el centro del núcleo
10	Células peritubulares o mioides	Agregaba filamentos de actina y miosina y capacidad contráctil, que no están en el manual
11	Intersticio testicular	Incluía vasos sanguíneos, que no figuran en arch4
21	Osteona	Daba una definición que el manual no tiene; el manual solo la nombra como estructura a identificar

Las referencias se reescribieron con lo que dice el manual, y las páginas esperadas incluyen ahora la página con la descripción, no solo el rótulo de la imagen. Para que las corridas anteriores sigan siendo comparables, `juez.py --referencias-actuales` vuelve a juzgar solo las preguntas cuya referencia cambió y copia los demás juicios. Cada juicio guarda la referencia con la que se hizo, así que una corrección posterior (la de la #5) repite solo ese juicio. `baseline.py --recalcular-recall` recalcula el recall con las páginas nuevas sin correr el pipeline.

6.6 Resultados

Todas las corridas, medidas con las referencias y páginas corregidas. El v7 se descartó (sección 6.8):

	Cambio	Recall @5	Juez global	Corrección	Fidelidad
v2	Chunking fijo de 500 caracteres	0.867	0.773	3.72	4.47
v3	Chunking por oraciones y fichas	0.852	0.820	4.03	4.66
v4	Chunking por secciones	0.852	0.793	4.00	4.50
v5	Búsqueda coseno + BM25	0.912	0.898	4.50	4.94
v6	**Encabezados entre páginas**	**0.912**	**0.902**	**4.56**	**4.94**
v7	**Fragmentos vecinos (sección 6.8)**	**0.912**	**0.887**	**4.41**	**4.81**

De una corrida a otra, Qwen y el juez no responden siempre igual: el puntaje de una misma pregunta se mueve entre 0.05 y 0.15 aunque llegue el mismo contexto. Por eso las diferencias globales de 0.03 o menos (v3 a v4, v5 a v6) no se distinguen del ruido. La subida de v4 a v5 (+0.10) sí, y coincide con las preguntas cuyo fragmento empezó a llegar. El recall del v3 y el v4 queda por debajo del v2 por la #5: desde el v3, la página 2 de arch4, que describe el intersticio, no llega entre los 5 primeros. En el v6, el recall por fuente es 0.983 en arch2, 0.767 en arch3 y 0.810 en arch4, y el puntaje global del juez es 0.911, 0.878 y 0.896.

Las 6 preguntas que fallaban en el v2 quedaron resueltas:

#	Pregunta	Global v2	Global v6
27	Clasificación de neuronas por número de prolongaciones	0.00	0.89
31	Células ependimarias	0.10	1.00
0	Capas de la arteria muscular	0.35	1.00
15	Estructuras de la lámina de laringe 43	0.49	0.89
23	Función y ubicación de los osteoclastos	0.54	0.89
30	Función de los oligodendrocitos	0.54	1.00

6.7 GEPA sobre el Baseline v6

Con la recuperación nueva, GEPA se volvió a correr con la misma configuración que en la sección 5 (semilla 0, 100 evaluaciones y la misma división train/validación), sobre los contextos del baseline v6 y con las referencias corregidas: `gepa.py` usa ahora la referencia actual del golden set y no la que quedó guardada en el baseline. En train quedaron 4 de las 5 preguntas flojas (#4, #9, #18 y #24) y en validación, la #5.

El primer intento se cortó a los dos minutos. Las tres keys chocaron seguido con el límite de 8.000 tokens por minuto del juez, y el rotador dejaba de reintentar a las cuatro rondas, así que `gepa.py` lo tomó como cuota agotada y no guardó nada. Con la espera ampliada (sección 2.2), la corrida completa usó 86 evaluaciones en 7 minutos. Tampoco esta vez hubo un prompt mejor:

Programa	Puntaje en validación (0–1)
Semilla (prompt actual)	**0.964**
Candidato 5	0.956
Candidato 1	0.944
Candidato 2	0.934
Candidato 4	0.919
Candidato 3	0.866

Las razones son las mismas que en la primera corrida, ahora más marcadas:

- La semilla saca 0.964, y 9 de las 10 preguntas de validación están entre 0.89 y 1.0. La diferencia con el candidato 5 (0.008) es menor que lo que mueve una sola pregunta.
- La única pregunta floja de validación, la #5 (0.79), falla por recuperación: la página que describe el intersticio no llega al contexto. Ningún candidato la superó.
- Los candidatos agregaron reglas de "cobertura total" y ejemplos armados con preguntas de train. Mejoraban los lotes de 3 preguntas de train, pero en validación siempre bajaban algo.

`PROMPTS_OPTIMIZADOS` sigue en `false`: con la recuperación actual, el prompt ya no es lo que limita el puntaje.

6.8 Fragmentos Vecinos (Baseline v7)

Las dos preguntas que fallan por recuperación tienen la respuesta en fragmentos contiguos al que sí llega: en la #5 sigue en la página 2 de arch4, y en la #24 está repartida en los fragmentos siguientes de la página 17 de arch2. Para el v7, `expandir_con_vecinos` (`src/qdrant_store.py`) suma al contexto los fragmentos anteriores y siguientes de cada resultado, hasta 2.500 caracteres extra. Los vecinos de los mejores resultados entran primero, la expansión no cruza a otra ficha o práctica, y los pasajes que se tocan se unen para no repetir texto. Antes de correrlo, `recuperacion.py` midió la fracción de palabras de la referencia que llega al contexto: subió de 0.804 a 0.820. El recall no cambia (0.912), porque los resultados son los mismos.

El juez dio 0.887, contra 0.902 del v6: el v7 fue mejor en 10 preguntas, peor en 12 e igual en 10. La diferencia global está dentro del ruido, pero ninguna de las preguntas a las que apuntaba mejoró:

- La #5 y la #24 quedaron igual (0.79 y 0.61). En la #5, el segundo resultado es de tejido óseo (arch2, página 7) y se llevó tres vecinos. Al pasaje del testículo se le sumó la túnica albugínea, pero no llegó a la página 2: siguen faltando el intersticio y las células de Leydig y de Sertoli. En la #24 entró completo el músculo estriado esquelético, pero no el cardíaco ni el liso.
- El presupuesto se reparte entre los 6 resultados, y varios son de otro tema. Con BM25 solo, una simulación previa daba una subida de 0.785 a 0.839; con los embeddings del pipeline fue de 0.804 a 0.820.
- Más contexto también trae más inferencias. En la #21, Qwen armó con el texto extra una definición de la osteona que el manual no da, y el puntaje bajó a 0.28.
- Subió solo el estilo docente (3.63 → 3.78), porque las respuestas explican más. Bajaron la corrección (4.56 → 4.41) y la fidelidad (4.94 → 4.81).

`VECINOS_EXTRA_CHARS` quedó en 0 por defecto, así que el pipeline funciona como en el v6. El código queda para otras pruebas, y `recuperacion.py --vecinos N` mide la cobertura de cualquier presupuesto sin gastar cuota.

7. Limitaciones

En el v6, 5 preguntas siguen por debajo de 0.8 (con la referencia corregida de la #5):

#	Pregunta	Global v6	Causa
24	Tres tipos de tejido muscular y sus características	0.61	Búsqueda: la respuesta está repartida en tres fragmentos, uno por tipo, y ninguno repite "tipos"
4	Corte transversal de una arteria muscular pequeña	0.70	Respuesta incompleta: omite el endotelio y la lámina elástica interna
9	Espermatogonia A oscura	0.71	Respuesta incompleta: el contexto tiene todo, pero Qwen responde en una oración
18	Cartílago hialino del disco de crecimiento	0.75	Respuesta incompleta: omite la laminilla (Hueso fracturado 79.c)
5	Organización del tejido testicular	0.79	Búsqueda: la página 2, que describe el intersticio, no llega entre los 5 primeros; la respuesta cubre solo la página 1

La #24 y la #5 fallan por recuperación, y sumar los fragmentos vecinos (sección 6.8) no las resolvió. En las otras tres, el contexto trae la respuesta y Qwen la resume de más; la segunda corrida de GEPA (sección 6.7) probó instrucciones que piden cubrir todo el contexto, y ninguna mejoró en validación. Además, el estilo docente sigue siendo la dimensión más baja (3.63 sobre 5).

8. Organización del Repositorio

El trabajo de la etapa está en la rama `feat/dspy-gepa`. Los scripts de evaluación y optimización están en `app/optimizacion/`:

```
app/optimizacion/
├── modelos.py       # lista y prueba los modelos de Groq
├── baseline.py      # pipeline completo sobre el golden set
├── juez.py          # LLM como juez con la rúbrica docente
├── gepa.py          # optimización del prompt con DSPy GEPA
├── comparar.py      # compara dos corridas pregunta por pregunta
├── inspeccionar.py  # contexto recuperado y juicio por pregunta
├── recuperacion.py  # recall de la búsqueda según el peso coseno/BM25
└── resultados/      # JSONL de cada corrida y logs de GEPA
```

La secuencia de comandos para reproducir la etapa, desde `app/`:

```
uv run python -m optimizacion.modelos --probar
uv run python -m optimizacion.baseline --salida
optimizacion/resultados/baseline-v2.jsonl
uv run python -m optimizacion.juez optimizacion/resultados/baseline-v2.jsonl
uv run python -m optimizacion.comparar <corrida-A>-juez.jsonl <corrida-B>-
juez.jsonl
uv run python -m optimizacion.gepa optimizacion/resultados/baseline-v6.jsonl
--max-metric-calls 100
uv run python -m optimizacion.inspeccionar <corrida>.jsonl 1 5 15
uv run python -m optimizacion.recuperacion --baseline <corrida>.jsonl
```

La recuperación nueva está en `app/src/chunking.py` (división en fragmentos), `app/src/lexico.py` (BM25) y `busqueda_densa_lexica` en `app/src/qdrant_store.py`. Los fragmentos vecinos están en `expandir_con_vecinos`, en el mismo archivo, y se activan con `VECINOS_EXTRA_CHARS`.

9. Conclusiones y Próximos Pasos

La Etapa 2 dejó una evaluación reproducible de punta a punta: el baseline mide la recuperación, el juez mide las respuestas y GEPA prueba variantes del prompt con la misma métrica. GEPA no mejoró el prompt y mostró que el límite estaba en la recuperación. Con el chunking por oraciones y secciones y la búsqueda coseno + BM25, el recall subió de 0.867 a 0.912 y el puntaje del juez de 0.773 a 0.902 (con las referencias corregidas). Las 6 preguntas que fallaban en el v2 sacan ahora 0.89 o más. Una segunda corrida de GEPA sobre el v6 tampoco superó al prompt actual, y sumar los fragmentos vecinos al contexto (v7) no mejoró las dos preguntas que fallan por recuperación. El v6 queda como versión final de la etapa.

Estado de las tareas de la etapa:

Tarea	Estado	Notas
Migración de modelos	Completada	qwen3.8-27b y gpt-oss-120b
Baseline reproducible	Completada	Recall fuente+página@5 = 0.852 sobre 32 preguntas
LLM como juez	Completada	Puntaje global 0.729 → 0.788
Optimización con GEPA	Completada, sin mejora	Dos corridas (v2 y v6): la semilla es el mejor programa (0.922 y 0.964); no se activa
Mejora de la recuperación	Completada	Recall 0.867 → 0.912; juez 0.773 → 0.902
Fragmentos vecinos	Completada, sin mejora	v7: juez 0.887 contra 0.902; desactivado por defecto
Corrección del golden set	Completada	5 referencias alineadas con el manual (#5, #9, #10, #11, #21)

Próximos pasos:

- Ampliar el golden set, sobre todo en arch3 y arch4 (5 y 7 preguntas). Con 10 preguntas de validación, cada una pesa un 10 % y las diferencias entre prompts de GEPA no se distinguen del ruido.
- Probar un modelo de embeddings multilingüe en lugar de MiniLM, entrenado en inglés, para que el coseno aporte en preguntas con vocabulario distinto del manual. Es lo que queda por probar para la #5 y la #24, que fallan por recuperación.
- Si la cátedra provee respuestas de referencia propias, volver a correr GEPA con `--ejemplos-docente`. El estilo docente es la dimensión más baja (3.63 sobre 5) y es la que un prompt puede mejorar.